FreeRTOS (Supplemental Features)

[]

ESP-IDF provides multiple features to supplement the features offered by FreeRTOS. These supplemental features are available on all FreeRTOS implementations supported by ESP-IDF (i.e., ESP-IDF FreeRTOS and Amazon SMP FreeRTOS). This document describes these supplemental features and is split into the following sections:

Contents

- FreeRTOS (Supplemental Features)
  - Overview
  - Ring Buffers
  - ESP-IDF Tick and Idle Hooks
  - TLSP Deletion Callbacks
  - IDF Additional API
  - Component Specific Properties
  - API Reference

Overview

ESP-IDF adds various new features to supplement the capabilities of FreeRTOS as follows:

- **Ring buffers**: Ring buffers provide a FIFO buffer that can accept entries of arbitrary lengths.
- **ESP-IDF Tick and Idle Hooks**: ESP-IDF provides multiple custom tick interrupt hooks and idle task hooks that are more numerous and more flexible when compared to FreeRTOS tick and idle hooks.
- **Thread Local Storage Pointer (TLSP) Deletion Callbacks**: TLSP Deletion callbacks are run automatically when a task is deleted, thus allowing users to clean up their TLSPs automatically.
- **IDF Additional API**: ESP-IDF specific functions added to augment the features of FreeRTOS.
- **Component Specific Properties**: Currently added only one component specific property `ORIG_INCLUDE_PATH`.

Ring Buffers

FreeRTOS provides stream buffers and message buffers as the primary mechanisms to send arbitrarily sized data between tasks and ISRs. However, FreeRTOS stream buffers and message buffers have the following limitations:

- Strictly single sender and single receiver
- Data is passed by copy
- Unable to reserve buffer space for a deferred send (i.e., send acquire)

Therefore, ESP-IDF provides a separate ring buffer implementation to address the issues above.

ESP-IDF ring buffers are strictly FIFO buffers that supports arbitrarily sized items. Ring buffers are a more memory efficient alternative to FreeRTOS queues in situations where the size of items is variable. The capacity of a ring buffer is not measured by the number of items it can store, but rather by the amount of memory used for storing items.

The ring buffer provides APIs to send an item, or to allocate space for an item in the ring buffer to be filled manually by the user. For efficiency reasons, **items are always retrieved from the ring buffer by reference**. As a result, all retrieved items **must also be returned** to the ring buffer by using `vRingbufferReturnItem()` or `vRingbufferReturnItemFromISR()`, in order for them to be removed from the ring buffer completely.

The ring buffers are split into the three following types:

No-Split buffers guarantee that an item is stored in contiguous memory and does not attempt to split an item under any circumstances. Use No-Split buffers when items must occupy contiguous memory. **Only this buffer type allows reserving buffer space for deferred sending.** Refer to the documentation of the functions `xRingbufferSendAcquire()` and `xRingbufferSendComplete()` for more details.

Allow-Split buffers allow an item to be split in two parts when wrapping around the end of the buffer if there is enough space at the tail and the head of the buffer combined to store the item. Allow-Split buffers are more memory efficient than No-Split buffers but can return an item in two parts when retrieving.

Byte buffers do not store data as separate items. All data is stored as a sequence of bytes, and any number of bytes can be sent or retrieved each time. Use byte buffers when separate items do not need to be maintained, e.g., a byte stream.

Note

No-Split buffers and Allow-Split buffers always store items at 32-bit aligned addresses. Therefore, when retrieving an item, the item pointer is guaranteed to be 32-bit aligned. This is useful especially when you need to send some data to the DMA.

Note

Each item stored in No-Split or Allow-Split buffers **requires an additional 8 bytes for a header**. Item sizes are also rounded up to a 32-bit aligned size, i.e., multiple of 4 bytes. However the true item size is recorded within the header. The sizes of No-Split and Allow-Split buffers will also be rounded up when created.

Usage

The following example demonstrates the usage of `xRingbufferCreate()` and `xRingbufferSend()` to create a ring buffer and then send an item to it:

```
#include "freertos/ringbuf.h"
static char tx_item[] = "test_item";

...

    //Create ring buffer
    RingbufHandle_t buf_handle;
    buf_handle = xRingbufferCreate(1028, RINGBUF_TYPE_NOSPLIT);
    if (buf_handle == NULL) {
        printf("Failed to create ring buffer\n");
    }

    //Send an item
    UBaseType_t res =  xRingbufferSend(buf_handle, tx_item, sizeof(tx_item),
pdMS_TO_TICKS(1000));
    if (res != pdTRUE) {
        printf("Failed to send item\n");
    }
```

The following example demonstrates the usage of `xRingbufferSendAcquire()` and `xRingbufferSendComplete()` instead of `xRingbufferSend()` to acquire memory on the ring buffer (of type `RINGBUF_TYPE_NOSPLIT`) and then send an item to it. This adds one more step, but allows getting the address of the memory to write to, and writing to the memory yourself.

```
#include "freertos/ringbuf.h"
#include "soc/lldesc.h"

typedef struct {
    lldesc_t dma_desc;
    uint8_t buf[1];
} dma_item_t;

#define DMA_ITEM_SIZE(N) (sizeof(lldesc_t)+(((N)+3)&(~3)))

...

    //Retrieve space for DMA descriptor and corresponding data buffer
    //This has to be done with SendAcquire, or the address may be different when we
copy
    dma_item_t *item;
    UBaseType_t res =  xRingbufferSendAcquire(buf_handle,
                       (void**) &item, DMA_ITEM_SIZE(buffer_size),
pdMS_TO_TICKS(1000));
    if (res != pdTRUE) {
        printf("Failed to acquire memory for item\n");
    }
    item->dma_desc = (lldesc_t) {
        .size = buffer_size,
        .length = buffer_size,
        .eof = 0,
        .owner = 1,
        .buf = item->buf,
    };
    //Actually send to the ring buffer for consumer to use
    res = xRingbufferSendComplete(buf_handle, (void *)item);
    if (res != pdTRUE) {
        printf("Failed to send item\n");
    }
```

The following example demonstrates retrieving and returning an item from a **No-Split ring buffer** using `xRingbufferReceive()` and `vRingbufferReturnItem()`

```
...

    //Receive an item from no-split ring buffer
    size_t item_size;
    char *item = (char *)xRingbufferReceive(buf_handle, &item_size,
pdMS_TO_TICKS(1000));

    //Check received item
    if (item != NULL) {
        //Print item
        for (int i = 0; i < item_size; i++) {
            printf("%c", item[i]);
        }
        printf("\n");
        //Return Item
        vRingbufferReturnItem(buf_handle, (void *)item);
    } else {
        //Failed to receive item
        printf("Failed to receive item\n");
    }
```

The following example demonstrates retrieving and returning an item from an **Allow-Split ring buffer** using `xRingbufferReceiveSplit()` and `vRingbufferReturnItem()`

```
...

    //Receive an item from allow-split ring buffer
    size_t item_size1, item_size2;
    char *item1, *item2;
    BaseType_t ret = xRingbufferReceiveSplit(buf_handle, (void **)&item1, (void
**)&item2, &item_size1, &item_size2, pdMS_TO_TICKS(1000));

    //Check received item
    if (ret == pdTRUE && item1 != NULL) {
        for (int i = 0; i < item_size1; i++) {
            printf("%c", item1[i]);
        }
        vRingbufferReturnItem(buf_handle, (void *)item1);
        //Check if item was split
        if (item2 != NULL) {
            for (int i = 0; i < item_size2; i++) {
                printf("%c", item2[i]);
            }
            vRingbufferReturnItem(buf_handle, (void *)item2);
        }
        printf("\n");
    } else {
        //Failed to receive item
        printf("Failed to receive item\n");
    }
```

The following example demonstrates retrieving and returning an item from a **byte buffer** using `xRingbufferReceiveUpTo()` and `vRingbufferReturnItem()`

```
...

    //Receive data from byte buffer
    size_t item_size;
    char *item = (char *)xRingbufferReceiveUpTo(buf_handle, &item_size,
pdMS_TO_TICKS(1000), sizeof(tx_item));

    //Check received data
    if (item != NULL) {
        //Print item
        for (int i = 0; i < item_size; i++) {
            printf("%c", item[i]);
        }
        printf("\n");
        //Return Item
        vRingbufferReturnItem(buf_handle, (void *)item);
    } else {
        //Failed to receive item
        printf("Failed to receive item\n");
    }
```

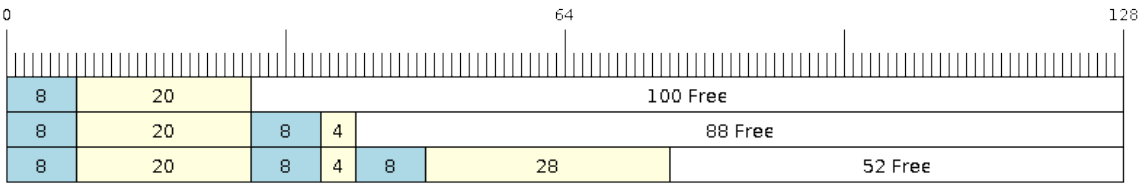
For ISR safe versions of the functions used above, call `xRingbufferSendFromISR()`, `xRingbufferReceiveFromISR()`, `xRingbufferReceiveSplitFromISR()`, `xRingbufferReceiveUpToFromISR()`, and `vRingbufferReturnItemFromISR()`.

Note

Two calls to `RingbufferReceive[UpTo][FromISR]()` are required if the bytes wraps around the end of the ring buffer.

Sending to Ring Buffer

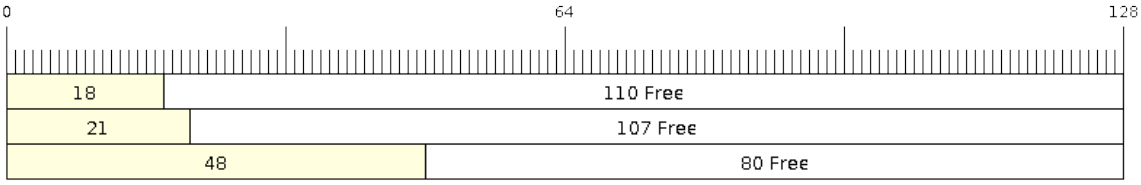
The following diagrams illustrate the differences between No-Split and Allow-Split buffers as compared to byte buffers with regard to sending items or data. The diagrams assume that three items of sizes **18, 3, and 27 bytes** are sent respectively to a **buffer of 128 bytes**:



Sending items to No-Split or Allow-Split ring buffers

For No-Split and Allow-Split buffers, a header of 8 bytes precedes every data item. Furthermore, the space occupied by each item is **rounded up to the nearest 32-bit aligned size** in order to maintain overall 32-bit alignment. However, the true size of the item is recorded inside the header which will be returned when the item is retrieved.

Referring to the diagram above, the 18, 3, and 27 byte items are **rounded up to 20, 4, and 28 bytes** respectively. An 8 byte header is then added in front of each item.



Sending items to byte buffers

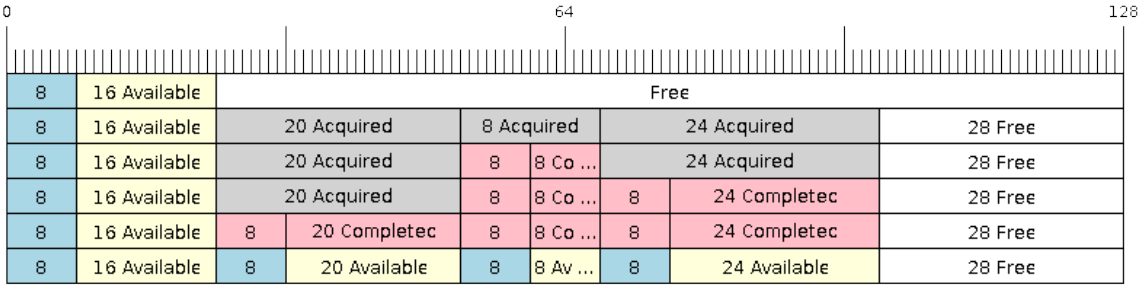
Byte buffers treat data as a sequence of bytes and does not incur any overhead (no headers). As a result, all data sent to a byte buffer is merged into a single item.

Referring to the diagram above, the 18, 3, and 27 byte items are sequentially written to the byte buffer and **merged into a single item of 48 bytes**.

Using SendAcquire and SendComplete

Items in No-Split buffers are acquired (by `SendAcquire`) in strict FIFO order and must be sent to the buffer by `SendComplete` for the data to be accessible by the consumer. Multiple items can be sent or acquired without calling `SendComplete`, and the items do not necessarily need to be completed in the order they were acquired. However, the receiving of data items must occur in FIFO order, therefore not calling `SendComplete` for the earliest acquired item prevents the subsequent items from being received.

The following diagrams illustrate what will happen when `SendAcquire` and `SendComplete` do not happen in the same order. At the beginning, there is already a data item of 16 bytes sent to the ring buffer. Then `SendAcquire` is called to acquire space of 20, 8, 24 bytes on the ring buffer.



SendAcquire/SendComplete items in No-Split ring buffers

After that, we fill (use) the buffers, and send them to the ring buffer by `SendComplete` in the order of 8, 24, 20. When 8 bytes and 24 bytes data are sent, the consumer still can only get the 16 bytes data item. Hence, if `SendComplete` is not called for the 20 bytes, it will not be

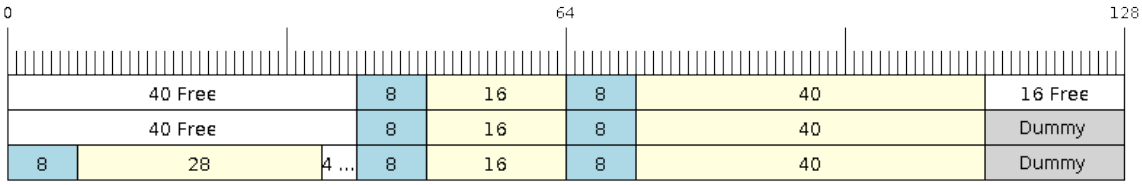
available, nor will the data items following the 20 bytes item.

When the 20 bytes item is finally completed, all the 3 data items can be received now, in the order of 20, 8, 24 bytes, right after the 16 bytes item existing in the buffer at the beginning.

Allow-Split buffers and byte buffers do not allow using `SendAcquire` or `SendComplete` since acquired buffers are required to be complete (not wrapped).

Wrap Around

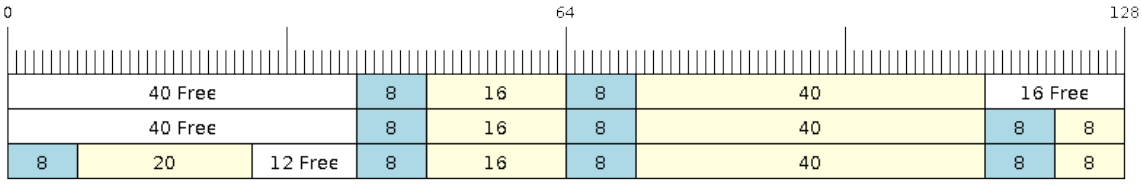
The following diagrams illustrate the differences between No-Split, Allow-Split, and byte buffers when a sent item requires a wrap around. The diagrams assume a buffer of **128 bytes** with **56 bytes of free space that wraps around** and a sent item of **28 bytes**.



Wrap around in No-Split buffers

No-Split buffers **only store an item in continuous free space and do not split an item under any circumstances**. When the free space at the tail of the buffer is insufficient to completely store the item and its header, the free space at the tail will be **marked as dummy data**. The buffer will then wrap around and store the item in the free space at the head of the buffer.

Referring to the diagram above, the 16 bytes of free space at the tail of the buffer is insufficient to store the 28 byte item. Therefore, the 16 bytes is marked as dummy data and the item is written to the free space at the head of the buffer instead.

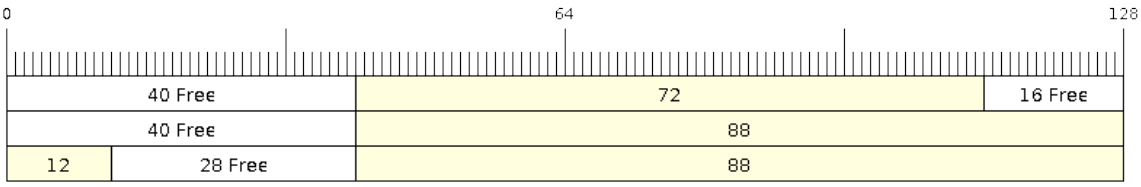


Wrap around in Allow-Split buffers

Allow-Split buffers will attempt to **split the item into two parts** when the free space at the tail of the buffer is insufficient to store the item data and its header. Both parts of the split item will have their own headers, therefore incurring an extra 8 bytes of overhead.

Referring to the diagram above, the 16 bytes of free space at the tail of the buffer is insufficient to store the 28 byte item. Therefore, the item is split into two parts (8 and 20 bytes) and written as two parts to the buffer.

> **Note**
>
> Allow-Split buffers treat both parts of the split item as two separate items, therefore call `xRingbufferReceiveSplit()` instead of `xRingbufferReceive()` to receive both parts of a split item in a thread safe manner.



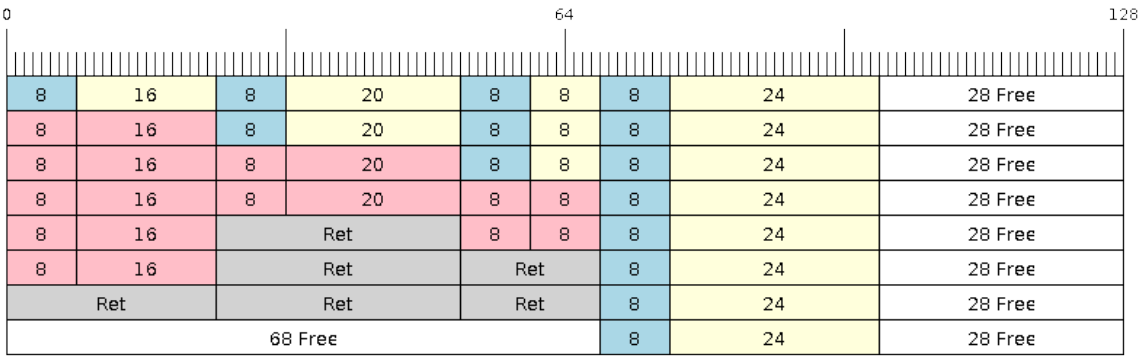
Wrap around in byte buffers

Byte buffers **store as much data as possible into the free space at the tail of buffer**. The remaining data will then be stored in the free space at the head of the buffer. No overhead is incurred when wrapping around in byte buffers.

Referring to the diagram above, the 16 bytes of free space at the tail of the buffer is insufficient to completely store the 28 bytes of data. Therefore, the 16 bytes of free space is filled with data, and the remaining 12 bytes are written to the free space at the head of the buffer. The buffer now contains data in two separate continuous parts, and each continuous part is treated as a separate item by the byte buffer.

Retrieving/Returning

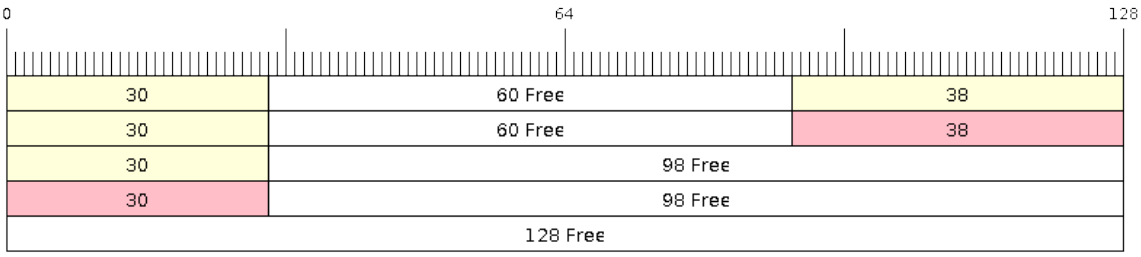
The following diagrams illustrate the differences between No-Split and Allow-Split buffers as compared to byte buffers in retrieving and returning data:



Retrieving/Returning items in No-Split and Allow-Split ring buffers

Items in No-Split buffers and Allow-Split buffers are **retrieved in strict FIFO order** and **must be returned** for the occupied space to be freed. Multiple items can be retrieved before returning, and the items do not necessarily need to be returned in the order they were retrieved. However, the freeing of space must occur in FIFO order, therefore not returning the earliest retrieved item prevents the space of subsequent items from being freed.

Referring to the diagram above, the **16, 20, and 8 byte items are retrieved in FIFO order**. However, the items are not returned in the order they were retrieved. First, the 20 byte item is returned followed by the 8 byte and the 16 byte items. The space is not freed until the first item, i.e., the 16 byte item is returned.



Retrieving/Returning data in byte buffers

Byte buffers **do not allow multiple retrievals before returning** (every retrieval must be followed by a return before another retrieval is permitted). When using `xRingbufferReceive()` or `xRingbufferReceiveFromISR()`, all continuous stored data will be retrieved. `xRingbufferReceiveUpTo()` or `xRingbufferReceiveUpToFromISR()` can be used to restrict the maximum number of bytes retrieved. Since every retrieval must be followed by a return, the space is freed as soon as the data is returned.

Referring to the diagram above, the 38 bytes of continuous stored data at the tail of the buffer is retrieved, returned, and freed. The next call to `xRingbufferReceive()` or `xRingbufferReceiveFromISR()` then wraps around and does the same to the 30 bytes of continuous stored data at the head of the buffer.

Note

Retrieving items from Allow-Split buffers must be done via `xRingbufferReceiveSplit()` or `xRingbufferReceiveSplitFromISR()` instead of `xRingbufferReceive()` or `xRingbufferReceiveFromISR()`.

Ring Buffers with Queue Sets

Ring buffers can be added to FreeRTOS queue sets using `xRingbufferAddToQueueSetRead()` such that every time a ring buffer receives an item or data, the queue set is notified. Once added to a queue set, every attempt to retrieve an item from a ring buffer should be

preceded by a call to `xQueueSelectFromSet()`. To check whether the selected queue set member is the ring buffer, call `xRingbufferCanRead()`.

The following example demonstrates queue set usage with ring buffers:

```
#include "freertos/queue.h"
#include "freertos/ringbuf.h"

...

    //Create ring buffer and queue set
    RingbufHandle_t buf_handle = xRingbufferCreate(1028, RINGBUF_TYPE_NOSPLIT);
    QueueSetHandle_t queue_set = xQueueCreateSet(3);

    //Add ring buffer to queue set
    if (xRingbufferAddToQueueSetRead(buf_handle, queue_set) != pdTRUE) {
        printf("Failed to add to queue set\n");
    }

...

    //Block on queue set
    QueueSetMemberHandle_t member = xQueueSelectFromSet(queue_set,
pdMS_TO_TICKS(1000));

    //Check if member is ring buffer
    if (member != NULL && xRingbufferCanRead(buf_handle, member) == pdTRUE) {
        //Member is ring buffer, receive item from ring buffer
        size_t item_size;
        char *item = (char *)xRingbufferReceive(buf_handle, &item_size, 0);

        //Handle item
        ...

    } else {
        ...
    }
```

Ring Buffers with Static Allocation

The `xRingbufferCreateStatic()` can be used to create ring buffers with specific memory requirements (such as a ring buffer being allocated in external RAM). All blocks of memory used by a ring buffer must be manually allocated beforehand, then passed to the `xRingbufferCreateStatic()` to be initialized as a ring buffer. These blocks include the following:

- The ring buffer's data structure of type `StaticRingbuffer_t`.
- The ring buffer's storage area of size `xBufferSize`. Note that `xBufferSize` must be 32-bit aligned for No-Split and Allow-Split buffers.

The manner in which these blocks are allocated depends on the users requirements (e.g., all blocks being statically declared, or dynamically allocated with specific capabilities such as external RAM).

> **Note**
>
> When deleting a ring buffer created via `xRingbufferCreateStatic()`, the function `vRingbufferDelete()` will not free any of the memory blocks. This must be done manually by the user after `vRingbufferDelete()` is called.

The code snippet below demonstrates a ring buffer being allocated entirely in external RAM.

```
#include "freertos/ringbuf.h"
#include "freertos/semphr.h"
#include "esp_heap_caps.h"

#define BUFFER_SIZE     400      //32-bit aligned size
#define BUFFER_TYPE     RINGBUF_TYPE_NOSPLIT
...

//Allocate ring buffer data structure and storage area into external RAM
StaticRingbuffer_t *buffer_struct = (StaticRingbuffer_t
*)heap_caps_malloc(sizeof(StaticRingbuffer_t), MALLOC_CAP_SPIRAM);
uint8_t *buffer_storage = (uint8_t *)heap_caps_malloc(sizeof(uint8_t)*BUFFER_SIZE,
MALLOC_CAP_SPIRAM);

//Create a ring buffer with manually allocated memory
RingbufHandle_t handle = xRingbufferCreateStatic(BUFFER_SIZE, BUFFER_TYPE,
buffer_storage, buffer_struct);

...

//Delete the ring buffer after used
vRingbufferDelete(handle);

//Manually free all blocks of memory
free(buffer_struct);
free(buffer_storage);
```

ESP-IDF Tick and Idle Hooks

FreeRTOS allows applications to provide a tick hook and an idle hook at compile time:

- FreeRTOS tick hook can be enabled via the CONFIG_FREERTOS_USE_TICK_HOOK option. The application must provide the `void vApplicationTickHook( void )` callback.
- FreeRTOS idle hook can be enabled via the CONFIG_FREERTOS_USE_IDLE_HOOK option. The application must provide the `void vApplicationIdleHook( void )` callback.

However, the FreeRTOS tick hook and idle hook have the following draw backs:

- The FreeRTOS hooks are registered at compile time
- Only one of each hook can be registered
- On multi-core targets, the FreeRTOS hooks are symmetric, meaning each core's tick interrupt and idle tasks ends up calling the same hook

Therefore, ESP-IDF tick and idle hooks are provided to supplement the features of FreeRTOS tick and idle hooks. The ESP-IDF hooks have the following features:

- The hooks can be registered and deregistered at run-time
- Multiple hooks can be registered (with a maximum of 8 hooks of each type per core)
- On multi-core targets, the hooks can be asymmetric, meaning different hooks can be registered to each core

ESP-IDF hooks can be registered and deregistered using the following APIs:

- For tick hooks:
    - Register using `esp_register_freertos_tick_hook()` or `esp_register_freertos_tick_hook_for_cpu()`
    - Deregister using `esp_deregister_freertos_tick_hook()` or `esp_deregister_freertos_tick_hook_for_cpu()`
- For idle hooks:
    - Register using `esp_register_freertos_idle_hook()` or `esp_register_freertos_idle_hook_for_cpu()`
    - Deregister using `esp_deregister_freertos_idle_hook()` or `esp_deregister_freertos_idle_hook_for_cpu()`

Note

The tick interrupt stays active while the cache is disabled, therefore any tick hook (FreeRTOS or ESP-IDF) functions must be placed in internal RAM. Please refer to the SPI flash API documentation for more details.

TLSP Deletion Callbacks

Vanilla FreeRTOS provides a Thread Local Storage Pointers (TLSP) feature. These are pointers stored directly in the Task Control Block (TCB) of a particular task. TLSPs allow each task to have its own unique set of pointers to data structures. Vanilla FreeRTOS expects users to:

- set a task's TLSPs by calling `vTaskSetThreadLocalStoragePointer()` after the task has been created.
- get a task's TLSPs by calling `pvTaskGetThreadLocalStoragePointer()` during the task's lifetime.

- free the memory pointed to by the TLSPs before the task is deleted.

However, there can be instances where users may want the freeing of TLSP memory to be automatic. Therefore, ESP-IDF provides the additional feature of TLSP deletion callbacks. These user-provided deletion callbacks are called automatically when a task is deleted, thus allowing the TLSP memory to be cleaned up without needing to add the cleanup logic explicitly to the code of every task.

The TLSP deletion callbacks are set in a similar fashion to the TLSPs themselves.

- `vTaskSetThreadLocalStoragePointerAndDelCallback()` sets both a particular TLSP and its associated callback.
- Calling the Vanilla FreeRTOS function `vTaskSetThreadLocalStoragePointer()` simply sets the TLSP's associated Deletion Callback to *NULL*, meaning that no callback is called for that TLSP during task deletion.

When implementing TLSP callbacks, users should note the following:

- The callback **must never attempt to block or yield** and critical sections should be kept as short as possible.
- The callback is called shortly before a deleted task's memory is freed. Thus, the callback can either be called from `vTaskDelete()` itself, or from the idle task.

IDF Additional API

The freertos/esp_additions/include/freertos/idf_additions.h header contains FreeRTOS-related helper functions added by ESP-IDF. Users can include this header via `#include "freertos/idf_additions.h"`.

Component Specific Properties

Besides standard component variables that are available with basic cmake build properties, FreeRTOS component also provides arguments (only one so far) for simpler integration with other modules:

- *ORIG_INCLUDE_PATH* - contains an absolute path to freertos root include folder. Thus instead of *#include "freertos/FreeRTOS.h"* you can refer to headers directly: *#include "FreeRTOS.h"*.

API Reference

Ring Buffer API

Header File

- components/esp_ringbuf/include/freertos/ringbuf.h
- This header file can be included with:

```
#include "freertos/ringbuf.h"
```

- This header file is a part of the API provided by the `esp_ringbuf` component. To declare that your component depends on `esp_ringbuf`, add the following to your CMakeLists.txt:

```
REQUIRES esp_ringbuf
```

or

```
PRIV_REQUIRES esp_ringbuf
```

Functions

RingbufHandle_t **xRingbufferCreate**(size_t xBufferSize, RingbufferType_t xBufferType)

Create a ring buffer.

Note

xBufferSize of no-split/allow-split buffers will be rounded up to the nearest 32-bit aligned size.

Parameters:
- **xBufferSize** -- **[in]** Size of the buffer in bytes. Note that items require space for a header in no-split/allow-split buffers
- **xBufferType** -- **[in]** Type of ring buffer, see documentation.

Returns: A handle to the created ring buffer, or NULL in case of error.

RingbufHandle_t **xRingbufferCreateNoSplit**(size_t xItemSize, size_t xItemNum)

Create a ring buffer of type RINGBUF_TYPE_NOSPLIT for a fixed item_size.

This API is similar to xRingbufferCreate(), but it will internally allocate additional space for the headers.

Parameters:
- **xItemSize** -- **[in]** Size of each item to be put into the ring buffer
- **xItemNum** -- **[in]** Maximum number of items the buffer needs to hold simultaneously

Returns: A RingbufHandle_t handle to the created ring buffer, or NULL in case of error.

RingbufHandle_t **xRingbufferCreateStatic**(size_t xBufferSize, RingbufferType_t xBufferType, uint8_t *pucRingbufferStorage, StaticRingbuffer_t *pxStaticRingbuffer)

Create a ring buffer but manually provide the required memory.

Note

xBufferSize of no-split/allow-split buffers MUST be 32-bit aligned.

Parameters:
- **xBufferSize** -- **[in]** Size of the buffer in bytes.
- **xBufferType** -- **[in]** Type of ring buffer, see documentation
- **pucRingbufferStorage** -- **[in]** Pointer to the ring buffer's storage area. Storage area must have the same size as specified by xBufferSize
- **pxStaticRingbuffer** -- **[in]** Pointed to a struct of type StaticRingbuffer_t which will be used to hold the ring buffer's data structure

Returns: A handle to the created ring buffer

BaseType_t **xRingbufferSend**(RingbufHandle_t xRingbuffer, *const* void *pvItem, size_t xItemSize, TickType_t xTicksToWait)

Insert an item into the ring buffer.

Attempt to insert an item into the ring buffer. This function will block until enough free space is available or until it times out.

Note

For no-split/allow-split ring buffers, the actual size of memory that the item will occupy will be rounded up to the nearest 32-bit aligned size. This is done to ensure all items are always stored in 32-bit aligned fashion.

Note

For no-split/allow-split buffers, an xItemSize of 0 will result in an item with no data being set (i.e., item only contains the header). For byte buffers, an xItemSize of 0 will simply return pdTRUE without copying any data.

Parameters:

- **xRingbuffer** -- **[in]** Ring buffer to insert the item into
- **pvItem** -- **[in]** Pointer to data to insert. NULL is allowed if xItemSize is 0.
- **xItemSize** -- **[in]** Size of data to insert.
- **xTicksToWait** -- **[in]** Ticks to wait for room in the ring buffer.

Returns:

- pdTRUE if succeeded
- pdFALSE on time-out or when the data is larger than the maximum permissible size of the buffer

---

BaseType_t **xRingbufferSendFromISR**(RingbufHandle_t xRingbuffer, *const* void *pvItem, size_t xItemSize, BaseType_t *pxHigherPriorityTaskWoken)

Insert an item into the ring buffer in an ISR.

Attempt to insert an item into the ring buffer from an ISR. This function will return immediately if there is insufficient free space in the buffer.

Note

For no-split/allow-split ring buffers, the actual size of memory that the item will occupy will be rounded up to the nearest 32-bit aligned size. This is done to ensure all items are always stored in 32-bit aligned fashion.

Note

For no-split/allow-split buffers, an xItemSize of 0 will result in an item with no data being set (i.e., item only contains the header). For byte buffers, an xItemSize of 0 will simply return pdTRUE without copying any data.

Parameters:

- **xRingbuffer** -- **[in]** Ring buffer to insert the item into
- **pvItem** -- **[in]** Pointer to data to insert. NULL is allowed if xItemSize is 0.
- **xItemSize** -- **[in]** Size of data to insert.
- **pxHigherPriorityTaskWoken** -- **[out]** Value pointed to will be set to pdTRUE if the function woke up a higher priority task.

Returns:

- pdTRUE if succeeded
- pdFALSE when the ring buffer does not have space.

BaseType_t **xRingbufferSendAcquire**(RingbufHandle_t xRingbuffer, void **ppvItem, size_t xItemSize, TickType_t xTicksToWait)

Acquire memory from the ring buffer to be written to by an external source and to be sent later.

Attempt to allocate buffer for an item to be sent into the ring buffer. This function will block until enough free space is available or until it times out.

The item, as well as the following items `SendAcquire` or `Send` after it, will not be able to be read from the ring buffer until this item is actually sent into the ring buffer.

Note

Only applicable for no-split ring buffers now, the actual size of memory that the item will occupy will be rounded up to the nearest 32-bit aligned size. This is done to ensure all items are always stored in 32-bit aligned fashion.

Note

An xItemSize of 0 will result in a buffer being acquired, but the buffer will have a size of 0.

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer to allocate the memory
- **ppvItem** -- **[out]** Double pointer to memory acquired (set to NULL if no memory were retrieved)
- **xItemSize** -- **[in]** Size of item to acquire.
- **xTicksToWait** -- **[in]** Ticks to wait for room in the ring buffer.

Returns:
- pdTRUE if succeeded
- pdFALSE on time-out or when the data is larger than the maximum permissible size of the buffer

---

BaseType_t **`xRingbufferSendComplete`**(RingbufHandle_t xRingbuffer, void *pvItem)

Actually send an item into the ring buffer allocated before by `xRingbufferSendAcquire`.

Note

Only applicable for no-split ring buffers. Only call for items allocated by `xRingbufferSendAcquire`.

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer to insert the item into
- **pvItem** -- **[in]** Pointer to item in allocated memory to insert.

Returns:
- pdTRUE if succeeded
- pdFALSE if fail for some reason.

---

void ***`xRingbufferReceive`**(RingbufHandle_t xRingbuffer, size_t *pxItemSize, TickType_t xTicksToWait)

Retrieve an item from the ring buffer.

Attempt to retrieve an item from the ring buffer. This function will block until an item is available or until it times out.

Note

A call to vRingbufferReturnItem() is required after this to free the item retrieved.

Note

It is possible to receive items with a pxItemSize of 0 on no-split buffers.

Note

To retrieve an item from an allow-split buffer, use `xRingbufferReceiveSplit()` instead.

Parameters:

- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **pxItemSize** -- **[out]** Pointer to a variable to which the size of the retrieved item will be written.
- **xTicksToWait** -- **[in]** Ticks to wait for items in the ring buffer.

Returns:

- Pointer to the retrieved item on success; *pxItemSize filled with the length of the item.
- NULL on timeout, *pxItemSize is untouched in that case.

---

void ***xRingbufferReceiveFromISR**(RingbufHandle_t xRingbuffer, size_t *pxItemSize)

Retrieve an item from the ring buffer in an ISR.

Attempt to retrieve an item from the ring buffer. This function returns immediately if there are no items available for retrieval

Note

A call to vRingbufferReturnItemFromISR() is required after this to free the item retrieved.

Note

Byte buffers do not allow multiple retrievals before returning an item

Note

Two calls to RingbufferReceiveFromISR() are required if the bytes wrap around the end of the ring buffer.

Note

It is possible to receive items with a pxItemSize of 0 on no-split buffers.

Note

To retrieve an item from an allow-split buffer, use `xRingbufferReceiveSplitFromISR()` instead.

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **pxItemSize** -- **[out]** Pointer to a variable to which the size of the retrieved item will be written.

Returns:
- Pointer to the retrieved item on success; *pxItemSize filled with the length of the item.
- NULL when the ring buffer is empty, *pxItemSize is untouched in that case.

BaseType_t **xRingbufferReceiveSplit**(RingbufHandle_t xRingbuffer, void **ppvHeadItem, void **ppvTailItem, size_t *pxHeadItemSize, size_t *pxTailItemSize, TickType_t xTicksToWait)

Retrieve a split item from an allow-split ring buffer.

Attempt to retrieve a split item from an allow-split ring buffer. If the item is not split, only a single item is retried. If the item is split, both parts will be retrieved. This function will block until an item is available or until it times out.

Note

Call(s) to vRingbufferReturnItem() is required after this to free up the item(s) retrieved.

Note

This function should only be called on allow-split buffers

Note

It is possible to receive items with a pxItemSize of 0 on allow split buffers.

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **ppvHeadItem** -- **[out]** Double pointer to first part (set to NULL if no items were retrieved)
- **ppvTailItem** -- **[out]** Double pointer to second part (set to NULL if item is not split)

- **pxHeadItemSize** -- **[out]** Pointer to size of first part (unmodified if no items were retrieved)
- **pxTailItemSize** -- **[out]** Pointer to size of second part (unmodified if item is not split)
- **xTicksToWait** -- **[in]** Ticks to wait for items in the ring buffer.

Returns:

- pdTRUE if an item (split or unsplit) was retrieved
- pdFALSE when no item was retrieved

BaseType_t **xRingbufferReceiveSplitFromISR**(RingbufHandle_t xRingbuffer, void **ppvHeadItem, void **ppvTailItem, size_t *pxHeadItemSize, size_t *pxTailItemSize)

Retrieve a split item from an allow-split ring buffer in an ISR.

Attempt to retrieve a split item from an allow-split ring buffer. If the item is not split, only a single item is retried. If the item is split, both parts will be retrieved. This function returns immediately if there are no items available for retrieval

Note

Calls to vRingbufferReturnItemFromISR() is required after this to free up the item(s) retrieved.

Note

This function should only be called on allow-split buffers

Note

It is possible to receive items with a pxItemSize of 0 on allow split buffers.

Parameters:

- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **ppvHeadItem** -- **[out]** Double pointer to first part (set to NULL if no items were retrieved)
- **ppvTailItem** -- **[out]** Double pointer to second part (set to NULL if item is not split)
- **pxHeadItemSize** -- **[out]** Pointer to size of first part (unmodified if no items were retrieved)
- **pxTailItemSize** -- **[out]** Pointer to size of second part (unmodified if item is not split)

Returns:

- pdTRUE if an item (split or unsplit) was retrieved
- pdFALSE when no item was retrieved

---

void *__xRingbufferReceiveUpTo__(RingbufHandle_t xRingbuffer, size_t *pxItemSize, TickType_t xTicksToWait, size_t xMaxSize)

Retrieve bytes from a byte buffer, specifying the maximum amount of bytes to retrieve.

Attempt to retrieve data from a byte buffer whilst specifying a maximum number of bytes to retrieve. This function will block until there is data available for retrieval or until it times out.

Note

A call to vRingbufferReturnItem() is required after this to free up the data retrieved.

Note

This function should only be called on byte buffers

Note

Byte buffers do not allow multiple retrievals before returning an item

Note

Two calls to RingbufferReceiveUpTo() are required if the bytes wrap around the end of the ring buffer.

Parameters:

- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **pxItemSize** -- **[out]** Pointer to a variable to which the size of the retrieved item will be written.
- **xTicksToWait** -- **[in]** Ticks to wait for items in the ring buffer.
- **xMaxSize** -- **[in]** Maximum number of bytes to return.

Returns:

- Pointer to the retrieved item on success; *pxItemSize filled with the length of the item.
- NULL on timeout, *pxItemSize is untouched in that case.

---

void ***xRingbufferReceiveUpToFromISR**(RingbufHandle_t xRingbuffer, size_t *pxItemSize, size_t xMaxSize)

Retrieve bytes from a byte buffer, specifying the maximum amount of bytes to retrieve. Call this from an ISR.

Attempt to retrieve bytes from a byte buffer whilst specifying a maximum number of bytes to retrieve. This function will return immediately if there is no data available for retrieval.

Note

A call to vRingbufferReturnItemFromISR() is required after this to free up the data received.

Note

This function should only be called on byte buffers

Note

Byte buffers do not allow multiple retrievals before returning an item

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **pxItemSize** -- **[out]** Pointer to a variable to which the size of the retrieved item will be written.
- **xMaxSize** -- **[in]** Maximum number of bytes to return. Size of 0 simply returns NULL.

Returns:
- Pointer to the retrieved item on success; *pxItemSize filled with the length of the item.
- NULL when the ring buffer is empty, *pxItemSize is untouched in that case.

---

void **vRingbufferReturnItem**(RingbufHandle_t xRingbuffer, void *pvItem)

Return a previously-retrieved item to the ring buffer.

Note

If a split item is retrieved, both parts should be returned by calling this function twice

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer the item was retrieved from
- **pvItem** -- **[in]** Item that was received earlier

---

void **vRingbufferReturnItemFromISR**(RingbufHandle_t xRingbuffer, void *pvItem, BaseType_t *pxHigherPriorityTaskWoken)

Return a previously-retrieved item to the ring buffer from an ISR.

Note

If a split item is retrieved, both parts should be returned by calling this function twice

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer the item was retrieved from
- **pvItem** -- **[in]** Item that was received earlier

- **pxHigherPriorityTaskWoken** -- **[out]** Value pointed to will be set to pdTRUE if the function woke up a higher priority task.

esp_err_t **vRingbufferReset**(RingbufHandle_t xRingbuffer)

Reset a ring buffer back to its original empty state.

Parameters: **xRingbuffer** -- **[in]** Ring buffer to reset
Returns: ESP_ERR_INVALID_STATE if one or more items are not sent, completed or returned ESP_OK if the operation was successful

void **vRingbufferDelete**(RingbufHandle_t xRingbuffer)

Delete a ring buffer.

Note

This function will not deallocate any memory if the ring buffer was created using xRingbufferCreateStatic(). Deallocation must be done manually be the user.

Parameters: **xRingbuffer** -- **[in]** Ring buffer to delete

size_t **xRingbufferGetMaxItemSize**(RingbufHandle_t xRingbuffer)

Get maximum size of an item that can be placed in the ring buffer.

This function returns the maximum size an item can have if it was placed in an empty ring buffer.

Note

The max item size for a no-split buffer is limited to ((buffer_size/2)-header_size). This limit is imposed so that an item of max item size can always be sent to an empty no-split buffer regardless of the internal positions of the buffer's read/write/free pointers.

Parameters: **xRingbuffer** -- **[in]** Ring buffer to query
Returns: Maximum size, in bytes, of an item that can be placed in a ring buffer.

size_t **xRingbufferGetCurFreeSize**(RingbufHandle_t xRingbuffer)

Get current free size available for an item/data in the buffer.

This gives the real time free space available for an item/data in the ring buffer. This represents the maximum size an item/data can have if it was currently sent to the ring buffer.

Note

An empty no-split buffer has a max current free size for an item that is limited to ((buffer_size/2)-header_size). See API reference for xRingbufferGetMaxItemSize().

Parameters: **xRingbuffer** -- **[in]** Ring buffer to query

Returns: Current free size, in bytes, available for an entry

BaseType_t **xRingbufferAddToQueueSetRead**(RingbufHandle_t xRingbuffer, QueueSetHandle_t xQueueSet)

Add the ring buffer to a queue set. Notified when data has been written to the ring buffer.

This function adds the ring buffer to a queue set, thus allowing a task to block on multiple queues/ring buffers. The queue set is notified when the new data becomes available to read on the ring buffer.

Parameters:

- **xRingbuffer** -- **[in]** Ring buffer to add to the queue set
- **xQueueSet** -- **[in]** Queue set to add the ring buffer to

Returns:

- pdTRUE on success, pdFALSE otherwise

---

static inline BaseType_t **xRingbufferCanRead**(RingbufHandle_t xRingbuffer, QueueSetMemberHandle_t xMember)

Check if the selected queue set member is a particular ring buffer.

This API checks if queue set member returned from xQueueSelectFromSet() is a particular ring buffer. If so, this indicates the ring buffer has items waiting to be retrieved.

Parameters:

- **xRingbuffer** -- **[in]** Ring buffer to check
- **xMember** -- **[in]** Member returned from xQueueSelectFromSet

Returns:

- pdTRUE when selected queue set member is the ring buffer
- pdFALSE otherwise.

BaseType_t **xRingbufferRemoveFromQueueSetRead**(RingbufHandle_t xRingbuffer, QueueSetHandle_t xQueueSet)

Remove the ring buffer from a queue set.

This function removes a ring buffer from a queue set. The ring buffer must have been previously added to the queue set using xRingbufferAddToQueueSetRead().

Parameters:

- **xRingbuffer** -- **[in]** Ring buffer to remove from the queue set
- **xQueueSet** -- **[in]** Queue set to remove the ring buffer from

Returns:

- pdTRUE on success
- pdFALSE otherwise

void **vRingbufferGetInfo**(RingbufHandle_t xRingbuffer, UBaseType_t *uxFree, UBaseType_t *uxRead, UBaseType_t *uxWrite, UBaseType_t *uxAcquire, UBaseType_t *uxItemsWaiting)

Get information about ring buffer status.

Get information of a ring buffer's current status such as free/read/write/acquire pointer positions, and number of items waiting to be retrieved. Arguments can be set to NULL if they are not required.

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer handle
- **uxFree** -- **[out]** Pointer use to store free pointer position
- **uxRead** -- **[out]** Pointer use to store read pointer position
- **uxWrite** -- **[out]** Pointer use to store write pointer position
- **uxAcquire** -- **[out]** Pointer use to store acquire pointer position
- **uxItemsWaiting** -- **[out]** Pointer use to store number of items (bytes for byte buffer) waiting to be retrieved

void **xRingbufferPrintInfo**(RingbufHandle_t xRingbuffer)

Debugging function to print the internal pointers in the ring buffer.

Parameters: **xRingbuffer** -- Ring buffer to show

BaseType_t **xRingbufferGetStaticBuffer**(RingbufHandle_t xRingbuffer, uint8_t **ppucRingbufferStorage, StaticRingbuffer_t **ppxStaticRingbuffer)

Retrieve the pointers to a statically created ring buffer.

Parameters:
- **xRingbuffer** -- **[in]** Ring buffer
- **ppucRingbufferStorage** -- **[out]** Used to return a pointer to the queue's storage area buffer
- **ppxStaticRingbuffer** -- **[out]** Used to return a pointer to the queue's data structure buffer

Returns: pdTRUE if buffers were retrieved, pdFALSE otherwise.

RingbufHandle_t **xRingbufferCreateWithCaps**(size_t xBufferSize, RingbufferType_t xBufferType, UBaseType_t uxMemoryCaps)

Creates a ring buffer with specific memory capabilities.

This function is similar to xRingbufferCreate(), except that it allows the memory allocated for the ring buffer to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

A queue created using this function must only be deleted using vRingbufferDeleteWithCaps()

Parameters:
- **xBufferSize** -- **[in]** Size of the buffer in bytes
- **xBufferType** -- **[in]** Type of ring buffer, see documentation.

- **uxMemoryCaps** -- **[in]** Memory capabilities of the queue's memory (see esp_heap_caps.h)

Returns: Handle to the created ring buffer or NULL on failure.

---

void **vRingbufferDeleteWithCaps**(RingbufHandle_t xRingbuffer)

Deletes a ring buffer previously created using xRingbufferCreateWithCaps()

Parameters: **xRingbuffer** -- Ring buffer

Structures

---

struct **xSTATIC_RINGBUFFER**

Struct that is equivalent in size to the ring buffer's data structure.

The contents of this struct are not meant to be used directly. This structure is meant to be used when creating a statically allocated ring buffer where this struct is of the exact size required to store a ring buffer's control data structure.

Type Definitions

---

typedef void ***RingbufHandle_t**

Type by which ring buffers are referenced. For example, a call to xRingbufferCreate() returns a RingbufHandle_t variable that can then be used as a parameter to xRingbufferSend(), xRingbufferReceive(), etc.

---

typedef struct xSTATIC_RINGBUFFER **StaticRingbuffer_t**

Struct that is equivalent in size to the ring buffer's data structure.

The contents of this struct are not meant to be used directly. This structure is meant to be used when creating a statically allocated ring buffer where this struct is of the exact size required to store a ring buffer's control data structure.

Enumerations

---

enum **RingbufferType_t**

Values:

enumerator **RINGBUF_TYPE_NOSPLIT**

No-split buffers will only store an item in contiguous memory and will never split an item. Each item requires an 8 byte overhead for a header and will always internally occupy a 32-bit aligned size of space.

enumerator **RINGBUF_TYPE_ALLOWSPLIT**

Allow-split buffers will split an item into two parts if necessary in order to store it. Each item requires an 8 byte overhead for a header, splitting incurs an extra header. Each item will always internally occupy a 32-bit aligned size of space.

enumerator **RINGBUF_TYPE_BYTEBUF**

Byte buffers store data as a sequence of bytes and do not maintain separate items, therefore byte buffers have no overhead. All data is stored as a sequence of byte and any number of bytes can be sent or retrieved each time.

enumerator **RINGBUF_TYPE_MAX**

Hooks API

Header File

- components/esp_system/include/esp_freertos_hooks.h
- This header file can be included with:

```
#include "esp_freertos_hooks.h"
```

Functions

esp_err_t **esp_register_freertos_idle_hook_for_cpu**(esp_freertos_idle_cb_t new_idle_cb, UBaseType_t cpuid)

Register a callback to be called from the specified core's idle hook. The callback should return true if it should be called by the idle hook once per interrupt (or FreeRTOS tick), and return false if it should be called repeatedly as fast as possible by the idle hook.

Warning

Idle callbacks MUST NOT, UNDER ANY CIRCUMSTANCES, CALL A FUNCTION THAT MIGHT BLOCK.

Parameters:
- **new_idle_cb** -- **[in]** Callback to be called
- **cpuid** -- **[in]** id of the core

Returns:
- ESP_OK: Callback registered to the specified core's idle hook
- ESP_ERR_NO_MEM: No more space on the specified core's idle hook to register callback
- ESP_ERR_INVALID_ARG: cpuid is invalid

esp_err_t **esp_register_freertos_idle_hook**(esp_freertos_idle_cb_t new_idle_cb)

Register a callback to the idle hook of the core that calls this function. The callback should return true if it should be called by the idle hook once per interrupt (or FreeRTOS tick), and return false if it should be called repeatedly as fast as possible by the idle hook.

Warning

Idle callbacks MUST NOT, UNDER ANY CIRCUMSTANCES, CALL A FUNCTION THAT MIGHT BLOCK.

Parameters: **new_idle_cb** -- **[in]** Callback to be called

Returns:

- ESP_OK: Callback registered to the calling core's idle hook
- ESP_ERR_NO_MEM: No more space on the calling core's idle hook to register callback

---

esp_err_t **esp_register_freertos_tick_hook_for_cpu**(esp_freertos_tick_cb_t new_tick_cb, UBaseType_t cpuid)

Register a callback to be called from the specified core's tick hook.

Parameters:

- **new_tick_cb** -- **[in]** Callback to be called
- **cpuid** -- **[in]** id of the core

Returns:

- ESP_OK: Callback registered to specified core's tick hook
- ESP_ERR_NO_MEM: No more space on the specified core's tick hook to register the callback
- ESP_ERR_INVALID_ARG: cpuid is invalid

---

esp_err_t **esp_register_freertos_tick_hook**(esp_freertos_tick_cb_t new_tick_cb)

Register a callback to be called from the calling core's tick hook.

Parameters: **new_tick_cb** -- **[in]** Callback to be called

Returns:

- ESP_OK: Callback registered to the calling core's tick hook
- ESP_ERR_NO_MEM: No more space on the calling core's tick hook to register the callback

---

void **esp_deregister_freertos_idle_hook_for_cpu**(esp_freertos_idle_cb_t old_idle_cb, UBaseType_t cpuid)

Unregister an idle callback from the idle hook of the specified core.

Parameters:

- **old_idle_cb** -- **[in]** Callback to be unregistered
- **cpuid** -- **[in]** id of the core

---

void **esp_deregister_freertos_idle_hook**(esp_freertos_idle_cb_t old_idle_cb)

Unregister an idle callback. If the idle callback is registered to the idle hooks of both cores, the idle hook will be unregistered from both cores.

Parameters: **old_idle_cb** -- **[in]** Callback to be unregistered

---

void **esp_deregister_freertos_tick_hook_for_cpu**(esp_freertos_tick_cb_t old_tick_cb, UBaseType_t cpuid)

Unregister a tick callback from the tick hook of the specified core.

Parameters:
- **old_tick_cb** -- **[in]** Callback to be unregistered
- **cpuid** -- **[in]** id of the core

---

void **esp_deregister_freertos_tick_hook**(esp_freertos_tick_cb_t old_tick_cb)

Unregister a tick callback. If the tick callback is registered to the tick hooks of both cores, the tick hook will be unregistered from both cores.

Parameters: **old_tick_cb** -- **[in]** Callback to be unregistered

Type Definitions

---

typedef bool (***esp_freertos_idle_cb_t**)(void)

---

typedef void (***esp_freertos_tick_cb_t**)(void)

Additional API

Header File

- components/freertos/esp_additions/include/freertos/idf_additions.h
- This header file can be included with:

```
#include "freertos/idf_additions.h"
```

Functions

BaseType_t **xTaskCreatePinnedToCore**(TaskFunction_t pxTaskCode, *const* char **const* pcName, *const* uint32_t ulStackDepth, void **const* pvParameters, UBaseType_t uxPriority, TaskHandle_t **const* pxCreatedTask, *const* BaseType_t xCoreID)

Create a new task that is pinned to a particular core.

This function is similar to xTaskCreate(), but allows the creation of a pinned task. The task's pinned core is specified by the xCoreID argument. If xCoreID is set to tskNO_AFFINITY, then the task is unpinned and can run on any core.

Note

If (configNUMBER_OF_CORES == 1), setting xCoreID to tskNO_AFFINITY will be be treated as 0.

Parameters:
- **pxTaskCode** -- Pointer to the task entry function.
- **pcName** -- A descriptive name for the task.
- **ulStackDepth** -- The size of the task stack specified as the NUMBER OF BYTES. Note that this differs from vanilla FreeRTOS.
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run.
- **pxCreatedTask** -- Used to pass back a handle by which the created task can be referenced.
- **xCoreID** -- The core to which the task is pinned to, or tskNO_AFFINITY if the task has no core affinity.

Returns: pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

---

TaskHandle_t **xTaskCreateStaticPinnedToCore**(TaskFunction_t pxTaskCode, *const* char *const pcName, *const* uint32_t ulStackDepth, void *const pvParameters, UBaseType_t uxPriority, StackType_t *const puxStackBuffer, StaticTask_t *const pxTaskBuffer, *const* BaseType_t xCoreID)

Create a new static task that is pinned to a particular core.

This function is similar to xTaskCreateStatic(), but allows the creation of a pinned task. The task's pinned core is specified by the xCoreID argument. If xCoreID is set to tskNO_AFFINITY, then the task is unpinned and can run on any core.

Note

If (configNUMBER_OF_CORES == 1), setting xCoreID to tskNO_AFFINITY will be be treated as 0.

Parameters:
- **pxTaskCode** -- Pointer to the task entry function.
- **pcName** -- A descriptive name for the task.
- **ulStackDepth** -- The size of the task stack specified as the NUMBER OF BYTES. Note that this differs from vanilla FreeRTOS.
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run.
- **puxStackBuffer** -- Must point to a StackType_t array that has at least ulStackDepth indexes
- **pxTaskBuffer** -- Must point to a variable of type StaticTask_t, which will then be used to hold the task's data structures,

- **xCoreID** -- The core to which the task is pinned to, or tskNO_AFFINITY if the task has no core affinity.

Returns: The task handle if the task was created, NULL otherwise.

BaseType_t **xTaskGetCoreID**(TaskHandle_t xTask)

Get the current core ID of a particular task.

Helper function to get the core ID of a particular task. If the task is pinned to a particular core, the core ID is returned. If the task is not pinned to a particular core, tskNO_AFFINITY is returned.

If CONFIG_FREERTOS_UNICORE is enabled, this function simply returns 0.

[refactor-todo] See if this needs to be deprecated (IDF-8145)(IDF-8164)

Note

If CONFIG_FREERTOS_SMP is enabled, please call vTaskCoreAffinityGet() instead.

Note

In IDF FreerTOS when configNUMBER_OF_CORES == 1, this function will always return 0,

Parameters: **xTask** -- The task to query
Returns: The task's core ID or tskNO_AFFINITY

TaskHandle_t **xTaskGetCurrentTaskHandleForCore**(BaseType_t xCoreID)

Get the handle of the task currently running on a certain core.

Because of the nature of SMP processing, there is no guarantee that this value will still be valid on return and should only be used for debugging purposes.

[refactor-todo] See if this needs to be deprecated (IDF-8145)

Parameters: **xCoreID** -- The core to query
Returns: Handle of the current task running on the queried core

---

StackType_t ***xTaskGetStackStart**(TaskHandle_t xTask)

Returns the start of the stack associated with xTask.

Returns the lowest stack memory address, regardless of whether the stack grows up or down.

Parameters: **xTask** -- Handle of the task associated with the stack returned. Set xTask to NULL to return the stack of the calling task.
Returns: A pointer to the start of the stack.

void **vTaskSetThreadLocalStoragePointerAndDelCallback**(TaskHandle_t xTaskToSet, BaseType_t xIndex, void *pvValue, TlsDeleteCallbackFunction_t pvDelCallback)

Set local storage pointer and deletion callback.

Each task contains an array of pointers that is dimensioned by the configNUM_THREAD_LOCAL_STORAGE_POINTERS setting in FreeRTOSConfig.h. The kernel does not use the pointers itself, so the application writer can use the pointers for any purpose they wish.

Local storage pointers set for a task can reference dynamically allocated resources. This function is similar to vTaskSetThreadLocalStoragePointer, but provides a way to release these resources when the task gets deleted. For each pointer, a callback function can be set. This function will be called when task is deleted, with the local storage pointer index and value as arguments.

Parameters:

- **xTaskToSet** -- Task to set thread local storage pointer for
- **xIndex** -- The index of the pointer to set, from 0 to configNUM_THREAD_LOCAL_STORAGE_POINTERS - 1.
- **pvValue** -- Pointer value to set.
- **pvDelCallback** -- Function to call to dispose of the local storage pointer when the task is deleted.

BaseType_t **xTaskCreatePinnedToCoreWithCaps**(TaskFunction_t pvTaskCode, *const* char *const pcName, *const* configSTACK_DEPTH_TYPE usStackDepth, void *const pvParameters, UBaseType_t uxPriority, TaskHandle_t *const pvCreatedTask, *const* BaseType_t xCoreID, UBaseType_t uxMemoryCaps)

Creates a pinned task where its stack has specific memory capabilities.

This function is similar to xTaskCreatePinnedToCore(), except that it allows the memory allocated for the task's stack to have specific capabilities (e.g., MALLOC_CAP_SPIRAM).

However, the specified capabilities will NOT apply to the task's TCB as a TCB must always be in internal RAM.

Parameters:

- **pvTaskCode** -- Pointer to the task entry function
- **pcName** -- A descriptive name for the task
- **usStackDepth** -- The size of the task stack specified as the number of bytes
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run.
- **pvCreatedTask** -- Used to pass back a handle by which the created task can be referenced.
- **xCoreID** -- Core to which the task is pinned to, or tskNO_AFFINITY if unpinned.
- **uxMemoryCaps** -- Memory capabilities of the task stack's memory (see esp_heap_caps.h)

Returns: pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

---

static inline BaseType_t **xTaskCreateWithCaps**(TaskFunction_t pvTaskCode, *const* char *const pcName, configSTACK_DEPTH_TYPE usStackDepth, void *const pvParameters, UBaseType_t uxPriority, TaskHandle_t *pvCreatedTask, UBaseType_t uxMemoryCaps)

Creates a task where its stack has specific memory capabilities.

This function is similar to xTaskCreate(), except that it allows the memory allocated for the task's stack to have specific capabilities (e.g., MALLOC_CAP_SPIRAM).

However, the specified capabilities will NOT apply to the task's TCB as a TCB must always be in internal RAM.

Note

A task created using this function must only be deleted using vTaskDeleteWithCaps()

Parameters:

- **pvTaskCode** -- Pointer to the task entry function
- **pcName** -- A descriptive name for the task
- **usStackDepth** -- The size of the task stack specified as the number of bytes
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run.
- **pvCreatedTask** -- Used to pass back a handle by which the created task can be referenced.
- **uxMemoryCaps** -- Memory capabilities of the task stack's memory (see esp_heap_caps.h)

Returns: pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

---

void **vTaskDeleteWithCaps**(TaskHandle_t xTaskToDelete)

Deletes a task previously created using xTaskCreateWithCaps() or xTaskCreatePinnedToCoreWithCaps()

Note

It is recommended to use this API to delete tasks from another task's context, rather than self-deletion. When the task is being deleted, it is vital to ensure that it is not running on another core. This API must not be called from an interrupt context.

Parameters: **xTaskToDelete** -- A handle to the task to be deleted

QueueHandle_t **xQueueCreateWithCaps**(UBaseType_t uxQueueLength, UBaseType_t uxItemSize, UBaseType_t uxMemoryCaps)

Creates a queue with specific memory capabilities.

This function is similar to xQueueCreate(), except that it allows the memory allocated for the queue to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

A queue created using this function must only be deleted using vQueueDeleteWithCaps()

Parameters:
- **uxQueueLength** -- The maximum number of items that the queue can contain.
- **uxItemSize** -- The number of bytes each item in the queue will require.
- **uxMemoryCaps** -- Memory capabilities of the queue's memory (see esp_heap_caps.h)

Returns: Handle to the created queue or NULL on failure.

void **vQueueDeleteWithCaps**(QueueHandle_t xQueue)

Deletes a queue previously created using xQueueCreateWithCaps()

Parameters: **xQueue** -- A handle to the queue to be deleted.

static inline SemaphoreHandle_t **xSemaphoreCreateBinaryWithCaps**(UBaseType_t uxMemoryCaps)

Creates a binary semaphore with specific memory capabilities.

This function is similar to vSemaphoreCreateBinary(), except that it allows the memory allocated for the binary semaphore to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

A binary semaphore created using this function must only be deleted using vSemaphoreDeleteWithCaps()

Parameters: **uxMemoryCaps** -- Memory capabilities of the binary semaphore's memory (see esp_heap_caps.h)

Returns: Handle to the created binary semaphore or NULL on failure.

static inline SemaphoreHandle_t **xSemaphoreCreateCountingWithCaps**(UBaseType_t uxMaxCount, UBaseType_t uxInitialCount, UBaseType_t uxMemoryCaps)

Creates a counting semaphore with specific memory capabilities.

This function is similar to xSemaphoreCreateCounting(), except that it allows the memory allocated for the counting semaphore to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

A counting semaphore created using this function must only be deleted using vSemaphoreDeleteWithCaps()

Parameters:
- **uxMaxCount** -- The maximum count value that can be reached.
- **uxInitialCount** -- The count value assigned to the semaphore when it is created.
- **uxMemoryCaps** -- Memory capabilities of the counting semaphore's memory (see esp_heap_caps.h)

Returns: Handle to the created counting semaphore or NULL on failure.

---

static inline SemaphoreHandle_t **xSemaphoreCreateMutexWithCaps**(UBaseType_t uxMemoryCaps)

Creates a mutex semaphore with specific memory capabilities.

This function is similar to xSemaphoreCreateMutex(), except that it allows the memory allocated for the mutex semaphore to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

A mutex semaphore created using this function must only be deleted using vSemaphoreDeleteWithCaps()

Parameters: **uxMemoryCaps** -- Memory capabilities of the mutex semaphore's memory (see esp_heap_caps.h)

Returns: Handle to the created mutex semaphore or NULL on failure.

---

static inline SemaphoreHandle_t **xSemaphoreCreateRecursiveMutexWithCaps**(UBaseType_t uxMemoryCaps)

Creates a recursive mutex with specific memory capabilities.

This function is similar to xSemaphoreCreateRecursiveMutex(), except that it allows the memory allocated for the recursive mutex to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

A recursive mutex created using this function must only be deleted using vSemaphoreDeleteWithCaps()

Parameters: **uxMemoryCaps** -- Memory capabilities of the recursive mutex's memory (see esp_heap_caps.h)

Returns: Handle to the created recursive mutex or NULL on failure.

---

void **vSemaphoreDeleteWithCaps**(SemaphoreHandle_t xSemaphore)

Deletes a semaphore previously created using one of the xSemaphoreCreate...WithCaps() functions.

Parameters: **xSemaphore** -- A handle to the semaphore to be deleted.

---

static inline StreamBufferHandle_t **xStreamBufferCreateWithCaps**(size_t xBufferSizeBytes, size_t xTriggerLevelBytes, UBaseType_t uxMemoryCaps)

Creates a stream buffer with specific memory capabilities.

This function is similar to xStreamBufferCreate(), except that it allows the memory allocated for the stream buffer to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

A stream buffer created using this function must only be deleted using vStreamBufferDeleteWithCaps()

Parameters:

- **xBufferSizeBytes** -- The total number of bytes the stream buffer will be able to hold at any one time.
- **xTriggerLevelBytes** -- The number of bytes that must be in the stream buffer before unblocking
- **uxMemoryCaps** -- Memory capabilities of the stream buffer's memory (see esp_heap_caps.h)

Returns: Handle to the created stream buffer or NULL on failure.

---

static inline void **vStreamBufferDeleteWithCaps**(StreamBufferHandle_t xStreamBuffer)

Deletes a stream buffer previously created using xStreamBufferCreateWithCaps()

Parameters: **xStreamBuffer** -- A handle to the stream buffer to be deleted.

---

static inline MessageBufferHandle_t **xMessageBufferCreateWithCaps**(size_t xBufferSizeBytes, UBaseType_t uxMemoryCaps)

Creates a message buffer with specific memory capabilities.

This function is similar to xMessageBufferCreate(), except that it allows the memory allocated for the message buffer to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

A message buffer created using this function must only be deleted using vMessageBufferDeleteWithCaps()

Parameters:
- **xBufferSizeBytes** -- The total number of bytes (not messages) the message buffer will be able to hold at any one time.
- **uxMemoryCaps** -- Memory capabilities of the message buffer's memory (see esp_heap_caps.h)

Returns: Handle to the created message buffer or NULL on failure.

---

static inline void **vMessageBufferDeleteWithCaps**(MessageBufferHandle_t xMessageBuffer)

Deletes a stream buffer previously created using xMessageBufferCreateWithCaps()

Parameters: **xMessageBuffer** -- A handle to the message buffer to be deleted.

EventGroupHandle_t **xEventGroupCreateWithCaps**(UBaseType_t uxMemoryCaps)

Creates an event group with specific memory capabilities.

This function is similar to xEventGroupCreate(), except that it allows the memory allocated for the event group to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note

An event group created using this function must only be deleted using vEventGroupDeleteWithCaps()

Parameters: **uxMemoryCaps** -- Memory capabilities of the event group's memory (see esp_heap_caps.h)

Returns: Handle to the created event group or NULL on failure.

---

void **vEventGroupDeleteWithCaps**(EventGroupHandle_t xEventGroup)

Deletes an event group previously created using xEventGroupCreateWithCaps()

Parameters: **xEventGroup** -- A handle to the event group to be deleted.

Type Definitions

---

typedef void (***TlsDeleteCallbackFunction_t**)(int, void*)

Prototype of local storage pointer deletion callback.

Was this page helpful?